Campus-DDoS-SDN: un banco de pruebas basado en Python para la detección de ataques DDoS multivector en redes de campus definidas por software

Anna Olenka Calderón Leiva[a], Ricardo Alexander Llanos Lozano[a], Cristhian Emilio Martínez Casas[a,*] and Alessandro Piero Poves Martínez[a]

[a] *Universidad Nacional del Centro del Perú, , Huancayo, , , Peru*

ARTICLE INFO

Keywords:
Redes definidas por software (SDN)
Aprendizaje automático
detección de ataques DDoS
Mininet
clasificación de tráfico de red

ABSTRACT

Las redes definidas por software (SDN) representan una práctica común en sistemas de gestión de TI empresariales dada su simplificada gestión y facilidad en uso, pero también representa un riesgo latente de que al ser vulneradas, terminen por vulnerar todos los servicios que dependan de dicha red. Los estudios con esta clase de redes generalmente suelen usar tráfico sintético, el cual socava el valor de los datos al no representar de manera fidedigna el tráfico real; o, por otra parte, incluyen una cantidad mínima de vectores. El presente estudio solventa estas dificultades al, primero, capturar tráfico real en una universidad pública por medio de 4 vectores: TCP-SYN, HTTP, Slowloris e ICMP , a la par de realizar la simulación de los ataques DDoS en un entorno dockerizado que emula las redes SDN de la universidad. La clasificación del tráfico se realizó por medio de 3 algoritmos de clasificación: Naive Bayes, Random Forest y la Regresión Logística. La captura del tráfico se realizó en sesiones de 120s por cada SDN a lo largo de 5 iteraciones, contando, en última instancia con 230,692 paquetes de red. Los resultados indican que los métodos de clasificación tuvieron resultados óptimos, con un mínimo de 0.997 en el F1 macro por parte de la regresión logística, y un 0.999 para el modelo de random forest y Naive Bayes. El estudio cumple su cometido al generar un dataset de tráfico de red multiclasse, y al establecer medios de clasificación que pueden ser ajustados mediante los datos propuestos.

1. Introducción

Las redes definidas por software (SDN) separan el plano de control del plano de datos, concentrando la inteligencia de la red en un controlador lógicamente centralizado [1]. Esta centralización simplifica la gestión de la red, pero también convierte al controlador en un punto único de falla: un ataque de denegación de servicio distribuido (DDoS) que sature al controlador, a los switches, o a un servicio alojado puede degradar o tumbar todo el segmento de red que gobierna. Detectar estos ataques a tiempo, y separarlos de forma confiable de picos de tráfico legítimo, requiere datasets que sean a la vez representativos de despliegues SDN reales y etiquetados con precisión.

La mayoría de los datasets de detección de intrusiones disponibles públicamente, como NSL-KDD [6], CICIDS2017 [4] y CICDDoS2019 [5], no fueron capturados dentro de un entorno SDN: carecen de señales propias de OpenFlow, como la ocupación de la tabla de flujos o la carga del controlador, y sin embargo suelen reutilizarse fuera de ese contexto para entrenar clasificadores orientados a SDN. Trabajos recientes empezaron a cerrar esta brecha generando datasets de DDoS directamente dentro de bancos de pruebas Mininet/Ryu. Kumar y Gupta publicaron dos datasets de un solo vector construidos así: un dataset de inundación TCP-SYN [2] y, más recientemente, un dataset de inundación ICMP [3] en el que el tráfico normal se captura conectando un host de Mininet a una conexión real a internet y navegando sitios web reales (YouTube, Facebook, noticias, etc.), en vez de depender de generadores de tráfico sintético. Este enfoque de tráfico real aumenta el realismo, pero introduce dos limitaciones prácticas para un benchmark reutilizable: el dataset resultante queda atado a un solo vector de ataque, y no es reproducible entre ejecuciones, ya que el contenido y los tiempos capturados dependen de lo que el internet real ofrezca en el momento de la captura.

Este trabajo, Campus-DDoS-SDN, aborda ambas limitaciones dentro de un único banco de pruebas completamente reproducible. Se lanzan cuatro vectores de ataque (inundación TCP-SYN, inundación HTTP, Slowloris e inundación ICMP) desde hosts ya comprometidos contra una misma víctima compartida, un portal web, dentro de una topología Mininet/Ryu que imita un campus universitario: un esquema de switches core/DMZ más un laboratorio emulado por área académica. A diferencia de los datasets previos de un solo vector o de capturas totalmente separadas, los usuarios legítimos siguen navegando el portal durante todo el ataque, de modo que cada captura mezcla flujos benignos y maliciosos contra el mismo objetivo, tal como ocurriría en un incidente DDoS real. Las etiquetas de verdad fundamental se asignan por IP de origen en vez de por archivo de captura, y cada ejecución además registra el efecto del ataque sobre la víctima tal como lo percibe un cliente real (código de respuesta HTTP, latencia, conexiones activas), junto con indicadores a nivel de red (paquetes y bytes por segundo, tamaño de la tabla de flujos OpenFlow).

El entorno completo, incluyendo los generadores de tráfico, el portal web propio y el pipeline de etiquetado, se empaqueta como una imagen Docker para eliminar cualquier

cristhianmartinezcasas@gmail.com (C.E.M. Casas)

Table 1
Metadata del código.

Nr.	Descripción	Detalle
C1	Versión actual del código	v1.0
C2	Enlace permanente al repositorio de esta versión	https://github.com/CrisMC2/sdn-ddos-multiclass-detection
C3	Enlace permanente a la Reproducible Capsule	–
C4	Licencia legal del código	MIT
C5	Sistema de control de versiones usado	git
C6	Lenguajes, herramientas y servicios usados	Python, Bash, Mininet, Open vSwitch, Ryu, CICFlowMeter, Docker
C7	Requisitos de compilación, entornos y dependencias	Docker, Ubuntu 24.04; Python 3.9.25 y 3.12.3
C8	Enlace a documentación para desarrolladores	Ver `README.md` del repositorio
C9	Email de soporte	cristhianmartinezcasas@gmail.com

dependencia de una conexión a internet en vivo o de un sistema operativo de host especifico. El banco de pruebas incluye dos pipelines de detección complementarios: uno por flujo individual, basado en características CICFlowMeter, y uno por ventana de tiempo, que agrega métricas del portal y telemetría OpenFlow por segundo para permitir la detección temprana del ataque.

Los principales aportes de este trabajo son:

i. Un dataset de DDoS multiclase que cubre cuatro vectores de ataque contra una misma víctima dentro de una topología SDN realista y multisegmento, en vez de un solo tipo de ataque contra una red plana.

ii. Etiquetado por flujo y por IP de origen que mantiene mezclado el tráfico legítimo y malicioso dentro de la misma captura, más cercano a un escenario de ataque real que las capturas totalmente separadas usadas en trabajos previos.

iii. Métricas de impacto combinadas a nivel de red (estadísticas de flujo y paquetes vía CICFlowMeter) y a nivel de aplicación (estado HTTP, latencia, saturación de conexiones), capturadas automáticamente en cada ejecución.

iv. Un entorno Docker de un solo comando, completamente reproducible, que no depende de una conexión a internet en vivo, resolviendo la limitación de reproducibilidad de los datasets SDN basados en tráfico real.

v. Un clasificador de bosque aleatorio, entrenado sin usar la IP, junto con un pipeline preparado para mezclar capturas Wireshark de tráfico benigno de estudiantes (autorizadas) con los ataques solo simulados.

vi. Telemetría del controlador OpenFlow exportada en cada ejecución por una aplicación Ryu propia: instala flujos por IP de origen y sondea periódicamente las estadísticas de la tabla de flujos (paquetes, bytes, entradas activas), que el pipeline une con las métricas del portal por ventana de tiempo. Esto aporta las señales SDN específicas que los datasets no-SDN reutilizados no tienen.

vii. Un dataset por ventana de tiempo (una fila por segundo, con contadores del portal y telemetría SDN agregada) y un pipeline de detección temprana evaluado con validación cruzada leave-one-run-out sobre cinco ejecuciones independientes. El bosque aleatorio obtiene F1 macro = 0,999 $\pm$ 0,001 y F1 binario = 1,000 $\pm$ 0,000 (sin falsos positivos). Una ablación de grupos de características muestra que la telemetría SDN aporta +0,054 de F1 macro sobre usar solo los contadores de red, lo que evidencia una contribución complementaria de las señales OpenFlow a la detección temprana.

1.1. Problema

Una precisión alta obtenida con un solo tipo de ataque y capturas separadas no demuestra que el detector pueda reconocer tráfico malicioso cuando los usuarios legítimos siguen usando el mismo servicio. Además, una evaluación que depende de navegar internet en vivo cambia entre ejecuciones. Por ello, el problema abordado es cómo construir una base reproducible que mezcle tráfico benigno y cuatro ataques contra el mismo portal, y que permita probar el modelo en un laboratorio distinto de los usados para entrenarlo.

1.2. Pregunta, objetivo e hipótesis de trabajo

La pregunta es si las estadísticas de flujo, sin direcciones IP, puertos ni tiempo, permiten separar `BENIGN`, `synflood`, `httpflood`, `slowloris` e `icmpflood` en un área académica no vista. El objetivo general es implementar y evaluar ese banco en Docker. Los objetivos específicos son: (i) modelar una topología de campus; (ii) capturar tráfico mixto; (iii) convertir y etiquetar los pcap; (iv) medir el efecto sobre el portal y el contenedor; y (v) evaluar un bosque aleatorio con separación por laboratorio. La hipótesis de trabajo es que las características de tamaño, duración y tasa del flujo permiten lograr un F1 macro mayor a 0,90 en el laboratorio retenido. Esta hipótesis se contrasta primero en casa; la validación con tráfico real de la UNCP queda como fase propuesta y autorizada.

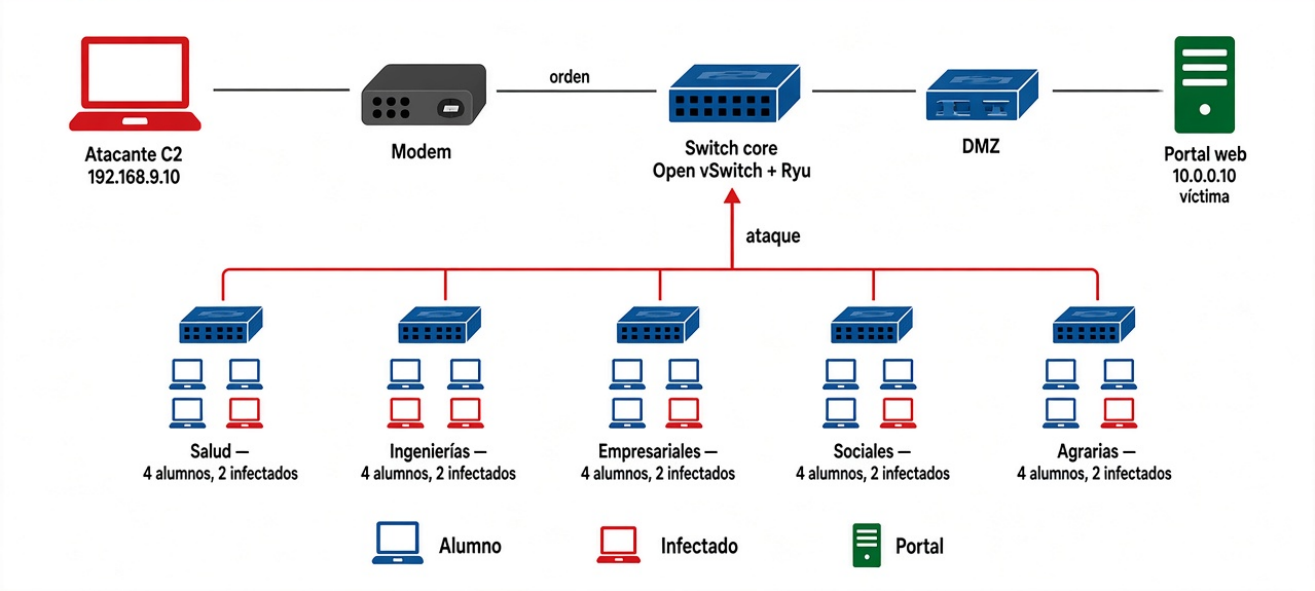


Figure 1: Topología emulada. El C2 queda fuera del campus; las PCs infectadas atacan el portal mientras los alumnos continúan navegando.

2. Flujo de trabajo

Todo el experimento se lanza con un solo comando sobre la imagen Docker `ataque-sdn`. Dentro del contenedor se levantan, en orden, el controlador Ryu, Open vSwitch y la topología Mininet; luego se ejecuta el escenario pedido y se recogen las salidas. El flujo completo va del campus emulado a los dos datasets etiquetados (por flujo y por ventana de tiempo) y a sus detectores (Fig. 2).

2.1. topología y población

La topología reproduce un campus reducido: un módem de borde, un switch *core*, una DMZ que aloja el portal web (la víctima, `10.0.0.10`) y un laboratorio por área académica, conectados por siete switches Open vSwitch gobernados por Ryu. Cada laboratorio tiene cuatro alumnos y dos PC comprometidas (20 alumnos y 10 PC infectadas en total). El centro de mando (C2) queda del lado de internet del módem (`192.168.9.10`); todo el campus vive en `10.0.0.0/8`. Durante un ataque los alumnos siguen pidiendo páginas del portal, y el navegador del operador entra como un cliente benigno más (`http://127.0.0.1:8080/` → `10.9.0.1`). El ataque no sale del contenedor.

2.2. Generación de tráfico mixto

Las PC comprometidas lanzan uno de cuatro vectores: inundación TCP-SYN (`hping3`), inundación HTTP, Slowloris (`slowhttptest`) e inundación ICMP. Los floods sostenidos (SYN e ICMP) corren en un bucle con `timeout` que relanza el generador cada 5 s para que el ritmo no decaiga; el ICMP usa el `ping` del kernel, que sostiene la tasa mejor que el modo `-icmp` de `hping3` sobre Open vSwitch. Como el tráfico legítimo nunca se detiene, cada captura mezcla flujos benignos y maliciosos contra el mismo portal. El banco se orquesta con `scripts/docker/run_scenario.sh`, que levanta un escenario o los cinco.

2.3. Captura y medición

Cada escenario deja, además del `pcap` tomado en la interfaz del portal, tres fuentes sincronizadas por segundo: una sonda del alumno (código HTTP, latencia y paquetes por segundo que percibe el portal), la telemetría del controlador OpenFlow, y el consumo del contenedor (CPU, memoria y red) vía `docker stats`. La telemetría SDN la produce una aplicación Ryu propia (`flow_monitor_13.py`) que, además de reenviar, instala un flujo por cada tripleta (IP origen, IP destino, protocolo) y muestrea periódicamente los contadores de esos flujos en el switch (paquetes, bytes y entradas activas). Los contadores se leen del plano de datos del switch para que reflejen el volumen real incluso bajo un flood sostenido, condición necesaria para que la señal de protocolo (en particular la de ICMP) no se pierda en la telemetría.

2.4. Etiquetado por IP de origen

La verdad fundamental se asigna por emisor, no por archivo de captura. Un flujo es `BENIGN` salvo que su IP de origen sea una PC comprometida y el escenario no sea el `baseline`; en ese caso lleva el nombre del ataque. Así el alumno, el portal y todo el `baseline` quedan `BENIGN` incluso dentro de una captura de ataque. Las IP comprometidas vienen en el JSON que acompaña a cada `pcap`.

2.5. Dos datasets, dos tareas

Del tráfico capturado se derivan dos conjuntos complementarios. *Por flujo* (`pcap_to_csv.py`): el tráfico TCP y UDP pasa por CICFlowMeter, que produce estadísticas bidireccionales; el ICMP, queno entra en ese filtro, lo cierra un agrupador propio a los 10 s (sin ese corte cada PC dejaría

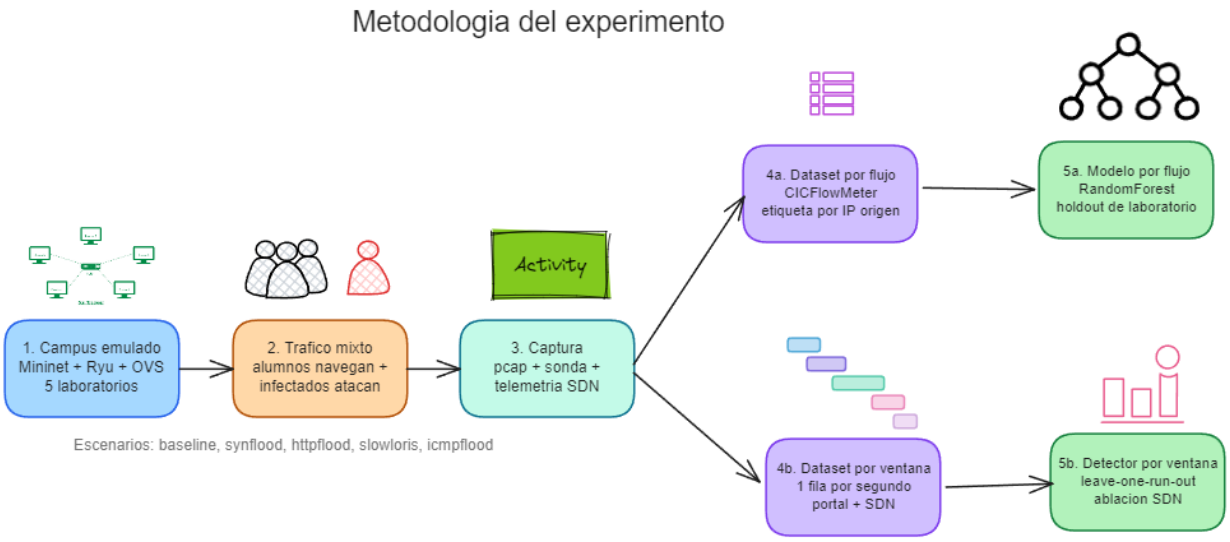


Figure 2: Pipeline del experimento: del campus emulado a los dos datasets (por flujo y por ventana) y sus detectores. La etiqueta se asigna por emisor, no por archivo.

un único flujo largísimo y la clase quedaría con muy pocas filas). A cada flujo se le añaden columnas `sdn_*` derivadas de la telemetría OpenFlow, cruzadas por IP de origen. Antes de entrenar se descartan la IP origen, la IP destino, los puertos y la marca de tiempo, para que el modelo aprenda comportamiento y no identidad de la máquina; los valores infinitos o vacíos pasan a cero y se quita toda columna constante. El modelo es un bosque de 200 árboles con peso por clase, porque la inundación SYN domina el conjunto. *Por ventana de tiempo* (`build_windows.py`): una fila por segundo que agrega los contadores del portal y la telemetría SDN del intervalo (entropía de las IP de origen, concentración del tráfico, crecimiento de la tabla de flujos y paquetes por protocolo), para la detección *mientras el ataque ocurre*.

2.6. Protocolo de evaluación

Nunca se usa una partición aleatoria, porque ventanas y flujos vecinos están correlacionados. El modelo por flujo (`analisis_completo.py`) se evalúa con un *holdout* de laboratorio: entrenan cuatro áreas académicas (185 127 flujos) y se prueba sobre una no vista, Ciencias Agrarias (45 550 flujos). El detector por ventana se evalúa con validación cruzada *leave-one-run-out* cuando hay dos o más ejecuciones. Para obtener media y desviación el banco ejecuta `run_scenario.sh repeat N`, que repite los cinco escenarios N veces con semillas e intensidades distintas (registradas en el `meta.json` de cada ejecución). Los resultados de este artículo salen de cinco ejecuciones de 120 s por escenario.

3. Requisitos de software y hardware

El banco corre en un contenedor Ubuntu 24.04; las versiones de la tabla se leyeron dentro de la imagen `ataque-sdn`. Los modelos se entrenan al correr los scripts de análisis (no son artefactos pre-entrenados): el detector reportado se obtuvo con Python 3.10.12, scikit-learn 0.23.2, pandas 1.3.5, numpy 1.21.5 y matplotlib 3.5.1. Ryu usa Python 3.9.25 en un entorno virtual separado.

Table 2
Requisitos del banco de pruebas.

N.°	Descripción	Herramienta	Versión
1	Emulador de red	Mininet	2.3.0
2	Switch virtual	Open vSwitch	3.3.9
3	Controlador SDN	Ryu	4.34
4	inundación SYN e ICMP	hping3	3.0.0-alpha-2
5	Ataque de baja tasa	slowhttptest	1.9.0
6	Captura de paquetes	tcpdump	4.99.4
7	pcap a CSV	CICFlowMeter	0.5.0
8	Sistema del contenedor	Ubuntu	24.04

Para repetir la ejecución basta una computadora de escritorio común, con 8 núcleos y 16 GB de RAM. Docker Desktop puede instalarse en Windows 11 o en Ubuntu. Al contenedor se le asignan 6 CPU y 8 GB, y dentro corre Ubuntu 24.04. Open vSwitch usa el *datapath* de kernel cuando el módulo `openvswitch` está disponible en el host, y cae automáticamente al *datapath* userspace (`netdev`) en caso contrario; la telemetría de flujo refleja correctamente los floods sostenidos en ambos modos.

4. Resultados y panorama de impacto

El banco es completamente reproducible: se repite con semillas controladas, mezcla alumno y ataque en la misma

Table 3
Sonda del portal (media de cinco corridas de 120 s por escenario).

Escenario	HTTP 200	Fallos	Latencia (ms)	Paquetes/s
baseline	120	0	402	145
synflood	120	0	401	715
httpflood	120	0	644	504
slowloris	1	111	1004	83
icmpflood	120	0	401	319

Table 4
Detección (bosque aleatorio). F1 macro 1,000.

Clase	Prec.	Recall	F1	Flujos	En el flujo
BENIGN	1,000	1,000	1,000	8769	se distingue bien
httpflood	1,000	1,000	1,000	7308	se distingue bien
icmpflood	1,000	1,000	1,000	129	se distingue bien
slowloris	1,000	1,000	1,000	708	se distingue bien
synflood	1,000	1,000	1,000	28636	se distingue bien

captura, y produce dos pipelines de detección complementarios. El clasificador por flujo alcanza un F1 macro de 1,000en el laboratorio de Ciencias Agrarias, retenido del entrenamiento. El detector por ventana, evaluado con leave-one-run-out sobre cinco ejecuciones independientes, obtiene F1 macro = 0,999 ± 0,001. Con autorización institucional, el mismo pipeline admite tráfico benigno real (Wireshark) sin sacar el ataque del contenedor.

4.1. Composición del dataset por flujo

Los CSV de las cinco ejecuciones suman 230 692 flujos: 143 133 `synflood`, 46 728 `BENIGN`, 36 595 `httpflood`, 3 598 `slowloris` y 638 `icmpflood`. Esta distribución explica la decisión de ponderar las clases durante el entrenamiento; a diferencia de ejecuciones previas, la clase ICMP reúne ahora suficientes flujos (129 en el laboratorio retenido) para informarse. Además, las capturas de ataque conservan una franja benigna, correspondiente a alumnos que siguen usando el portal.

4.2. Impacto sobre el portal

SYN genera la mayor tasa, 715 paquetes/s, pero la sonda mantiene las 120 respuestas HTTP 200. HTTP flood eleva la latencia media de 402 a 644 ms. Slowloris produce solo 83 paquetes/s, pero ocupa las conexiones disponibles: falla en 111 de 112 intentos. Por tanto, la tasa de paquetes por sí sola no describe el impacto de aplicación.

4.3. Rendimiento del clasificador por flujo

El F1 macro en Agrarias es 1,000y el F1 binario `BENIGN` frente a ataque es 1,000. Entre las variables de mayor peso aparecen tanto estadísticas de CICFlowMeter (la media y la variabilidad de la longitud de paquete de ida y el tamaño medio del segmento de ida) como columnas derivadas de la telemetría OpenFlow cruzada por IP de origen (`sdn_avg_pkt_bytes`, `sdn_bytes`, `sdn_pkts`). Esto es

Table 5
Comparación de modelos (leave-one-run-out, 5 folds).

Modelo	F1 macro	±	F1 bin.	±
Bosque aleatorio	0,999	0,001	1,000	0,000
Gradient boosting	0,999	0,001	1,000	0,000
Reg. logística	0,997	0,004	0,999	0,001

Table 6
Ablación de features (bosque aleatorio).

Grupo	F1 macro	±
Solo red (12 features)	0,945	0,104
Solo SDN (14 features)	0,892	0,115
Todas (26 features)	0,999	0,001

coherente con los escenarios –SYN genera segmentos pequeños y repetidos, HTTP intercambia solicitudes y respuestas completas, y Slowloris sostiene conexiones con poco tráfico– y muestra que las señales SDN aportan también a nivel de flujo individual.

4.4. detección temprana por ventana de tiempo

El dataset por ventana contiene 2 960 ventanas de un segundo, distribuidas en cinco ejecuciones independientes (r01–r05). La distribución por clase es prácticamente balanceada: 600 ventanas para BENIGN, httpflood, icmpflood y synflood, y 560 para slowloris (Fig. 3). Este balance es resultado del diseño experimental, donde cada escenario dura 120 s por ejecución.

La Fig. 4 muestra la distribución de la tasa de paquetes al portal por segundo en todas las ejecuciones. Synflood presenta la tasa más alta y consistente. Httpflood tiene una mediana menor pero con outliers extremos, reflejando la variabilidad en la intensidad entre ejecuciones. Slowloris es el más llamativo: su mediana es baja, similar al baseline, pero presenta outliers elevados; esto es coherente con su naturaleza de ataque de agotamiento de conexiones, node volumen. Esta dispersión confirma que la tasa de paquetes por sí sola no basta para distinguir todos los ataques, lo que motiva el uso de features combinadas de red, portal y telemetría SDN.

La Tabla 5 compara los tres modelos evaluados. Los tres superan ampliamente la hipótesis de trabajo (F1 macro > 0,90). El bosque aleatorio y el gradient boosting son equivalentes; la regresión logística, siendo un modelo lineal, muestra mayor varianza entre ejecuciones pero sigue siendo competitiva. Se selecciona el bosque aleatorio por su estabilidad y su capacidad de producir importancias de features interpretables.

La Tabla 6 y la Fig. 5 presentan la ablación de features. Las features de red solas alcanzan F1 = 0,945 con alta variabilidad (±0,104), indicando que algunos ataques son difíciles de separar usando solo contadores de paquetes y bytes. Las features SDN solas obtienen F1 = 0,892 (±0,115). La combinación de ambos grupos elimina casi toda la incertidumbre y eleva el F1 a 0,999 (±0,001). La telemetría

Table 7
Recall por clase del bosque aleatorio (todas las features).

Clase	Recall	±
BENIGN	1,000	0,000
httpflood	1,000	0,000
icmpflood	0,998	0,003
slowloris	0,996	0,004
synflood	1,000	0,000

OpenFlow aporta +0,054 de F1 macro sobre usar solo contadores de red, mostrando que las señales del controlador SDN tienen un rol complementario en la detección temprana.

La Tabla 7 detalla el recall por clase. El modelo no genera falsos positivos sobre tráfico benigno (0,0% ± 0,0%) en ninguna ejecución. Las dos clases con recall ligeramente inferior a 1 son icmpflood (una ventana clasificada como synflood) y slowloris (dos ventanas clasificadas como httpflood). El tiempo hasta la primera alarma es 0,0 s en todos los ataques: el modelo detecta el ataque en la primera ventana de medición.

La Fig. 6 muestra la matriz de confusión acumulada sobre los cinco folds. La diagonal es prácticamente perfecta: solo 3 errores sobre 2 960 ventanas.

La Fig. 7 presenta las diez features con mayor peso en el bosque. Las dos más discriminantes son contadores del portal: `conns_roll3` (media móvil de conexiones) y `core_flows` (entradas activas en la tabla de flujos del core), seguidas de métricas de servidor. A partir de la sexta posición aparecen features SDN (`sdn_pkts_tcp`, `sdn_pkts_icmp`), confirmando el aporte complementario de la telemetría OpenFlow.

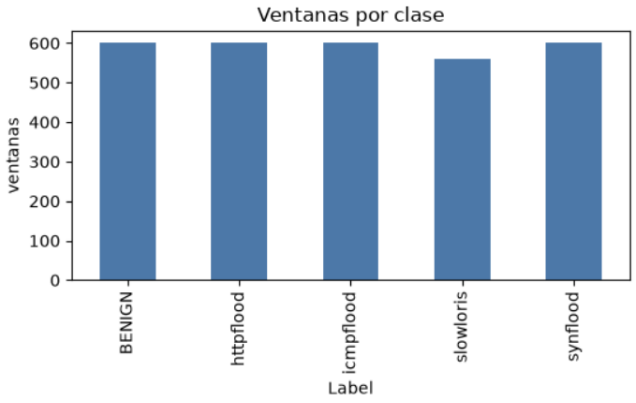


Figure 3: distribución de ventanas por clase en el dataset de detección temprana.

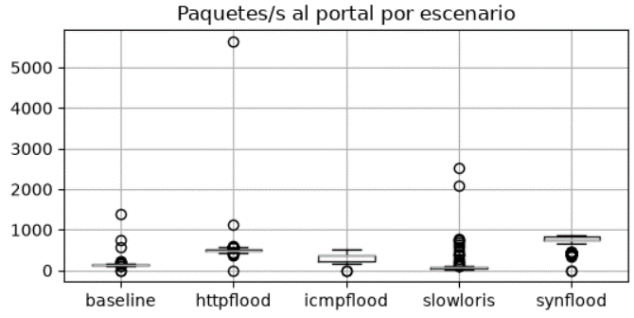


Figure 4: Tasa de paquetes por segundo al portal por escenario. Slowloris y baseline pueden ser indistinguibles en volumen.

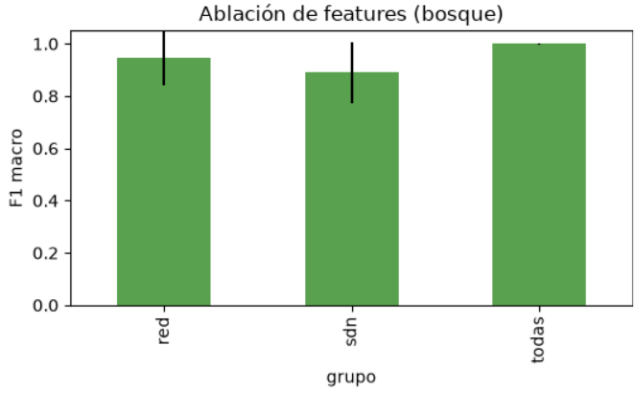


Figure 5: ablación de features: red, SDN y todas. Las barras de error indican la desviación estándar entre folds.

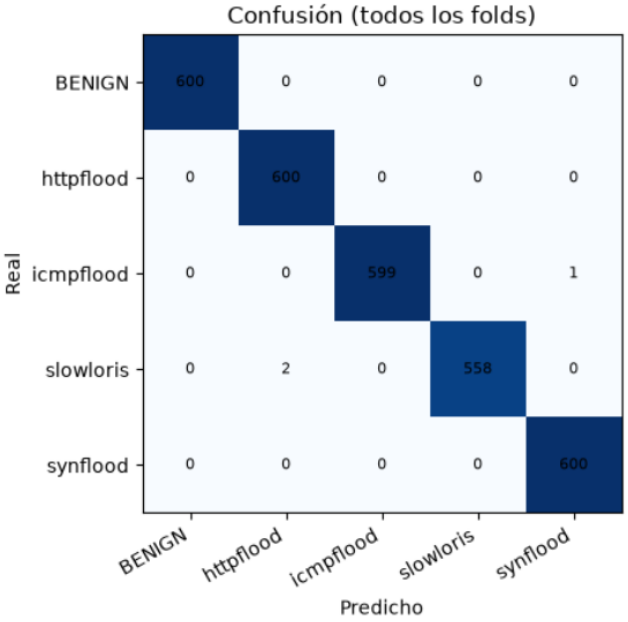


Figure 6: Matriz de confusión acumulada sobre los 5 folds. Los únicos errores son 1 ventana icmpflood → synflood y 2 ventanas slowloris → httpflood.

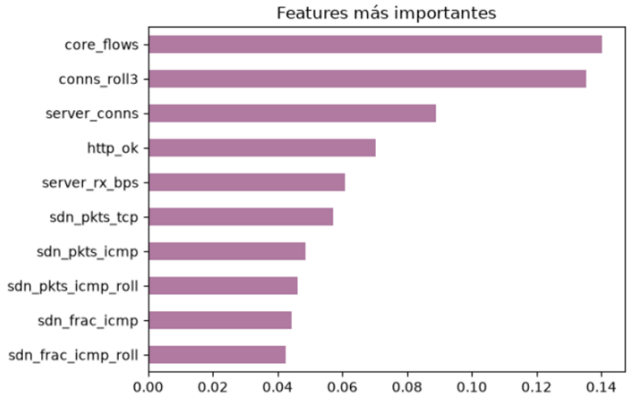


Figure 7: Diez features con mayor importancia en el bosque aleatorio.

4.5. Consumo del banco

La CPU media del contenedor fue 40% en baseline, 36% en SYN, 71% en HTTP, 313% en Slowloris y 35% en ICMP; en Docker, 100% equivale aproximadamente a un núcleo. Slowloris ocupó cerca de tres de los seis núcleos asignados, mientras la memoria máxima observada se mantuvo en torno a 230 MiB (en HTTP flood). Estos valores miden todo el banco (Mininet, OVS, Ryu, portal y generadores), no solo la víctima.

5. Discusión

La prueba respalda la hipótesis para las cinco clases: el modelo supera 0,90 de F1 macro sin IP y generaliza hacia un laboratorio retenido. El F1 macro de 0,999 del detector por ventana debe leerse con cautela y no como una afirmación de rendimiento en producción: a resolución de un segundo las ventanas de ataque presentan firmas muy marcadas frente al tráfico benigno, y la evaluación es de laboratorio (*leave-one-run-out* sobre cinco ejecuciones), no un despliegue en línea sobre una red real. El resultado más informativo no es la cifra absoluta sino la ablación: que la telemetría OpenFlow aporte +0,054 de F1 macro, y que sea decisiva para distinguir ICMP de SYN, es la evidencia concreta de que las señales propias de SDN tienen valor discriminante que los datasets no-SDN no ofrecen. La diferencia entre SYN y Slowloris también muestra por qué conviene unir clasificación de flujo con una sonda de servicio: más paquetes no implica necesariamente una página menos disponible.

5.1. Limitaciones y mejoras futuras

El resultado sale aún de alumnos emulados. El dataset híbrido (Wireshark + ataque simulado) queda listo en el código y se actualiza al recibir los pcap autorizados. Cada escenario dura 120 s. ICMP suma 638 flujos en el conjunto por flujo y 129 en la prueba de Agrarias; en el dataset por ventana, icmpflood tiene 600 ventanas y presentó 1 error de clasificación. El clasificador no corre dentro de Ryu; trabaja sobre datos ya capturados. Trabajo futuro: repetir la prueba con el fondo Wireshark, un escenario mixto de cuatro vectores a la vez, y despliegue en línea dentro del controlador.

6. Conclusiones

Campus-DDoS-SDN es un banco de pruebas reproducible, de un solo comando y sin dependencia de internet en vivo, que genera un dataset DDoS multiclase en el que el tráfico benigno y cuatro vectores de ataque conviven en la misma captura contra un único portal. El etiquetado por emisor conserva tráfico benigno dentro de la captura atacada, y la partición por laboratorio ofrece una prueba más exigente que un corte aleatorio. Sobre los flujos ya terminados, un bosque aleatorio entrenado sin IP, puertos ni tiempo alcanza un F1 macro de 1,000 en el laboratorio retenido de Ciencias Agrarias; sobre el dataset por ventana permite la detección temprana con F1 macro de 0,999, donde la telemetría OpenFlow aporta +0,054 y resulta decisiva para separar ICMP de SYN. Esto confirma el aporte central del trabajo: las señales propias de SDN, ausentes en los datasets no-SDN reutilizados, tienen valor discriminante medible.

Disponibilidad, tasa de paquetes y clasificación describen aspectos distintos del incidente: solo Slowloris deja el portal caído pese a su baja tasa de paquetes. El resultado, con alumnos emulados, es la base sobre la que se solicita autorización para incorporar tráfico benigno real de la UNCP, sin sacar el ataque del contenedor.

References

[1] Kreutz, D., Ramos, F.M.V., Veríssimo, P.E., Rothenberg, C.E., Azodolmolky, S., Uhlig, S., 2015. Software-defined networking: A comprehensive survey. Proceedings of the IEEE 103, 14–76. doi:10.1109/JPROC.2014.2371999.

[2] Kumar, S., Gupta, S., 2025. SDN TCP-SYN dataset: A dataset for TCP-SYN flood DDoS attack detection in software-defined networks. Data in Brief 59, 111314. doi:10.1016/j.dib.2025.111314.

[3] Kumar, S., Gupta, S., 2026. ICMP-Flood-SDN: A Python based machine learning application for ICMP flood DDoS attack detection in software defined networks. Software Impacts 27, 100819. doi:10.1016/j.simpa.2026.100819.

[4] Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A., 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), pp. 108–116. doi:10.5220/0006639801080116.

[5] Sharafaldin, I., Lashkari, A.H., Hakak, S., Ghorbani, A.A., 2019. Developing realistic distributed denial of service (DDoS) attack dataset and taxonomy, in: Proceedings of the 53rd IEEE International Carnahan Conference on Security Technology (ICCST), pp. 1–8. doi:10.1109/CCST.2019.8888419.

[6] Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A., 2009. A detailed analysis of the KDD CUP 99 data set, in: Proceedings of the 2nd IEEE International Conference on Computational Intelligence for Security and Defense Applications (CISDA).